Knowledge Engineering and Ontologies Course Project Report

Name & Email

Elay Yusifli 210315090

GitHub Repository: https://github.com/ElayYusifli/KnowoldgeProject

WIDOCO Documentation: https://elayyusifli.github.io/KnowoldgeProject/

Executive Summary

This project designs and implements a semantic knowledge engineering solution for academic course recommendation. The system models university courses, students, instructors, departments, topics, skills, prerequisites, and recommendations using an OWL ontology. A sample course catalog is transformed into RDF/Turtle with RDFlib, producing a knowledge graph that supports transparent SPARQL querying. The project includes SHACL validation rules to check required course metadata, credit ranges, course levels, and recommendation links. It also proposes an LLM integration workflow in which natural-language student questions are mapped to SPARQL queries and answered only with knowledge graph-grounded results. The main contribution of the project is an explainable course recommendation structure where each recommendation can be traced to student interests, completed prerequisites, missing requirements, and target skills.

Description of the Project

The project addresses the real-world problem of academic course selection. Students often need to choose courses based on interests, completed prerequisites, credit constraints, and desired skills, but this information is usually spread across course catalog pages and informal advisor knowledge. The objective is to create an ontology-driven knowledge graph that makes course information machine-interpretable and queryable. The system uses ontology engineering principles to define a conceptual schema in OWL, constructs RDF triples from structured CSV data using RDFlib, validates data quality with SHACL, and demonstrates semantic retrieval with SPARQL. The intended users are students, academic advisors, curriculum planners, and ontology engineering learners. The expected outputs are an OWL/Turtle ontology, RDF knowledge graph files, SPARQL queries, SHACL shapes, a reproducible Python construction script, WIDOCO documentation, and a public GitHub repository.

Phase 2 extends the project with research integration, a clearer data acquisition pipeline, and ontology versioning. The ontology version was updated from v0.1 to v0.2 by adding concepts for DataSource, ResearchStudy, and StudentBehaviorSignal. These additions make the project more explicit about provenance, research grounding, and student behavior modeling.

The project is guided by the following competency questions:

- Which courses are available in the catalog?
- Which courses match a student's interests?
- Which prerequisites are required for a selected course?
- Which prerequisites are missing for a recommended course?

- Which skills does each course develop?
- Which courses support a target skill such as SPARQL?
- How many courses exist at each academic level?
- Which courses has a student already completed?

Ontology Design

The ontology is named Smart Academic Course Recommendation Ontology and uses the namespace https://example.org/acr#. It models the academic advising domain with classes, object properties, and data properties. The main classes are Course, Student, Instructor, Department, Topic, Skill, and Recommendation.

The TBox contains the conceptual schema. Course represents a university course. Student represents a learner receiving recommendations. Instructor and Department represent organizational context. Topic describes the subject matter covered by a course, while Skill describes capabilities developed by the course. Recommendation is modeled as a separate class instead of a simple relationship so confidence scores and natural-language explanations can be attached.

The main object properties are offeredBy, taughtBy, hasPrerequisite, coversTopic, developsSkill, completedCourse, interestedIn, recommendedCourse, and recommendedFor. The main data properties are courseCode, courseTitle, credits, level, confidenceScore, and explanation.

In Phase 2, the ontology was expanded with DataSource, ResearchStudy, and StudentBehaviorSignal. The property derivedFromSource connects graph resources to their data origins, informedByStudy documents which research study influenced a component, and hasBehaviorSignal connects students to behavioral signals such as completed courses, stated interests, and target skills. This extension supports provenance and future adaptive recommendation logic.

The ABox contains instance-level data. For example, ex:course_CS101 is an instance of acr:Course, ex:department_Computer_Science is an instance of acr:Department, and ex:student_001 is an instance of acr:Student. The ABox also contains relationships such as ex:course_KE402 acr:hasPrerequisite ex:course_CS310 and ex:student_001 acr:interestedIn ex:topic_Ontologies.

Several modeling decisions were made deliberately. Courses, students, topics, and skills are modeled as classes because they describe reusable categories of entities. Individual course records are modeled as instances. Prerequisites are represented as course-to-course object properties because a prerequisite is a relationship between two course instances. Topics and skills are separated because a topic describes content coverage, while a skill describes learner capability. Recommendations are first-class resources because they need metadata such as confidence and explanation.

No external ontology was directly imported in this implementation. However, the design follows reusable Semantic Web vocabularies and modeling practices from RDF, RDFS, OWL, and SHACL. The extension methodology is closest to Modular Ontology Modeling because the Phase 2 additions form a small provenance and research-integration module without disrupting the course, student, and recommendation core.

Data Acquisition

The current dataset is a controlled sample course catalog stored in CSV format at data/sample/courses.csv. The dataset contains course code, title, credits, level, department, instructor, covered topics, developed skills, and prerequisites. The data is intentionally small so that the ontology, RDF conversion, SPARQL querying, and validation workflow can be reproduced easily.

The preprocessing process includes reading the CSV file, splitting semicolon-separated topic and skill values, normalizing labels into URI-safe resource names, creating reusable resources for departments, instructors, topics, and skills, and resolving prerequisite codes after all courses are loaded. Missing prerequisite values are treated as empty lists. Duplicate topics and skills are reused through the same generated resource URI. The main limitation is that the sample data is manually curated and small; a production version should use a larger public university course catalog and include stronger normalization of course descriptions.

The planned full data acquisition process has three source categories. The first source is a university course catalog, which provides course codes, titles, credits, descriptions, and prerequisites. The second source is department curriculum documentation, which supports mapping courses to academic levels and departments. The third source is manually curated skill and topic annotations, which are needed because catalogs usually describe content in natural language rather than formal skill entities. If the system is expanded, these sources can be collected through public HTML pages, PDF curriculum documents, or CSV exports from department pages.

The mapping from data to ontology concepts is direct. Course rows map to acr:Course; department values map to acr:Department; instructor values map to acr:Instructor; topic strings map to acr:Topic; skill strings map to acr:Skill; prerequisite course codes map to acr:hasPrerequisite; student profile examples map to acr:Student; and recommendation records map to acr:Recommendation. In Phase 2, the data acquisition design also maps catalog files and manual annotation files to acr:DataSource so that graph resources can be traced back to their origin.

Research Integration

For the optional research integration component, this project uses the study "Ontology-based question answering over corporate structured data" by Gorshkov, Kondratiev, and Shebalov. The study was selected because it directly connects ontology-based modeling, natural language understanding, SPARQL generation, and question answering over structured data. This is highly relevant to the course recommendation project because students and advisors should be able to ask questions in natural language, while the system should answer through the knowledge graph rather than through unsupported text generation.

The study is integrated into the proposed LLM layer and architecture. Its main idea is adapted as a controlled natural-language-to-SPARQL pipeline: the ontology provides the vocabulary, the knowledge graph provides the facts, and the LLM maps user intent to query templates. In this project, the integration point is the LLM workflow described later in the report. The model should identify course, topic, skill, prerequisite, and student profile entities, then generate or select one of the SPARQL queries in the repository. The result is more explainable than a free-form chatbot because every answer must be grounded in returned RDF triples.

Mandatory Paper Review

The mandatory course-provided study is titled "Personalized Ontology and Deep Training Tree-Based Optimal GRU-RNN for Prediction of Students' Behavior." Based on the title and assignment context, the study combines a personalized ontology with a deep learning sequence model for predicting student behavior. The ontology component likely represents learner-related concepts such as profile information, learning activity, performance indicators, behavior categories, and educational context. The GRU-RNN component is suitable for sequential student behavior data because gated recurrent units can model time-dependent patterns such as changing engagement, progress, or risk signals. The "deep training tree-based optimal" part suggests an optimization or feature selection layer used to improve training quality and prediction accuracy.

The key contribution of the study is the combination of symbolic and neural approaches. The ontology provides structured, interpretable educational knowledge, while the GRU-RNN model provides predictive capability over student behavior sequences. This neurosymbolic direction is relevant to the current project because course recommendation should not only retrieve courses but also explain why a course is appropriate. Similar ideas can be adapted by representing completed courses, interests, target skills, and missing prerequisites as StudentBehaviorSignal instances. A future version could use these signals as features for a predictive model that estimates course suitability or student success probability. The ontology would preserve explainability, while the model would support personalization.

Knowledge Graph Construction

The knowledge graph is built by combining the ontology schema with instance data. The Python script src/build_graph.py uses RDFlib to read data/sample/courses.csv and generate RDF/Turtle files. The generated data/sample/course_kg.ttl contains course catalog triples, while data/sample/full_kg.ttl combines course triples with student and recommendation examples from data/sample/students.ttl.

RDF represents knowledge as triples in the form subject-predicate-object. Representative triples from this project include:

- ex:course_CS101 rdf:type acr:Course
- ex:course_CS101 acr:courseTitle "Introduction to Programming"
- ex:course_CS201 acr:hasPrerequisite ex:course_CS101
- ex:course_KE402 acr:developsSkill ex:skill_SPARQL
- ex:student_001 acr:interestedIn ex:topic_Ontologies
- ex:rec_001 acr:recommendedCourse ex:course_KE402

The graph can be loaded into GraphDB by creating a repository, importing the Turtle files, and registering the acr and ex namespaces. The same graph can also be queried locally with RDFlib-compatible tools or any SPARQL endpoint that supports Turtle import.

SPARQL Queries

The repository includes ten SPARQL queries that correspond to the competency questions. Basic retrieval queries include list_courses.rq, courses_by_department.rq, course_skills.rq, and student_completed_courses.rq. Recommendation-oriented queries include recommend_courses.rq,

missing_prerequisites.rq, and courses_for_target_skill.rq. Reasoning-style traversal is demonstrated by prerequisite_chain.rq, which uses the property path acr:hasPrerequisite+ to retrieve direct and indirect prerequisites. Aggregation queries include count_courses_by_level.rq and count_skills_by_course.rq.

Example query for recommending courses based on interests and completed prerequisites:

sparql

```
PREFIX acr: <https://example.org/acr#>

PREFIX ex: <https://example.org/acr/resource/>

SELECT DISTINCT ?course ?title ?topic

WHERE {

ex:student_001 acr:interestedIn ?topic .

?course a acr:Course ;

acr:courseTitle ?title ;

acr:coversTopic ?topic .

FILTER NOT EXISTS {

?course acr:hasPrerequisite ?prerequisite .

FILTER NOT EXISTS { ex:student_001 acr:completedCourse ?prerequisite }

}

}

ORDER BY ?title
```

The expected output is a list of courses that match the student's interests and do not have unmet prerequisites. The missing prerequisite query is also important because it explains why a recommended course may not be immediately suitable. Aggregation queries provide overview statistics, such as how many courses exist at each level and how many skills are associated with each course.

Validation

The project uses SHACL to validate graph consistency and data quality. The SHACL file is located at shacl/course_shapes.ttl. CourseShape targets all instances of acr:Course and requires each course to have a course code, course title, credits, and level. It also restricts credits to the range 1-10 and course level to the range 1-4. RecommendationShape targets all instances of acr:Recommendation and requires each recommendation to point to a student and a course. It also checks that confidence scores are decimal values between 0 and 1.

Validation was executed with:

bash

```
python3 -m pyshacl -s shacl/course_shapes.ttl -d data/sample/full_kg.ttl -f human
```

The validation result was Conforms: True. During development, an initial validation issue occurred because recommendation data referenced generated course resources without loading the course graph. This was resolved by updating the graph construction script to generate full_kg.ttl, which combines course, student, and recommendation triples before validation.

LLM Integration

The project includes an LLM integration design for knowledge-grounded question answering. The LLM receives a natural-language question such as "Which course should I take if I am interested in ontologies?" The system prompt instructs the model to map the question to known ontology concepts, select or generate a SPARQL query, execute the query against the knowledge graph, and answer only from returned triples.

The workflow is:

- User asks a natural-language academic advising question.
- LLM identifies relevant entities such as student, topic, skill, or course.
- LLM selects a query template or generates a constrained SPARQL query.
- The SPARQL query is executed against the knowledge graph.
- The final answer is generated from query results with an explanation of prerequisites and skills.

Hallucination is reduced by grounding answers in SPARQL results, rejecting unsupported claims, exposing missing prerequisite explanations, and using SHACL validation to improve graph quality. Example interaction: if a student asks for ontology-related courses, the system can query courses covering ex:topic_Ontologies and then check whether required prerequisites are completed before recommending the course.

Evaluation, Discussion and Conclusion

The ontology is consistent with the intended domain and separates conceptual entities clearly. The distinction between topics and skills improves explainability, while modeling recommendations as resources allows the system to attach confidence and explanation metadata. The knowledge graph construction process is reproducible because data is stored in CSV and transformed with a deterministic RDFlib script. SPARQL queries demonstrate basic retrieval, recommendation filtering, prerequisite traversal, and aggregation. SHACL validation confirms that required data fields and recommendation links are structurally valid.

The main limitation is dataset size. The current data is a small curated sample rather than a full institutional course catalog. Another limitation is that topic and skill annotations are manually assigned, which may introduce subjectivity. The LLM integration is designed as a workflow rather than a fully deployed chatbot, so future work should implement a user interface, connect to a live SPARQL endpoint, and evaluate generated SPARQL queries with test questions.

In conclusion, the project demonstrates how ontology engineering and Semantic Web technologies can support transparent academic course recommendation. The system shows how OWL modeling, RDF graph construction, SPARQL querying, SHACL validation, and LLM grounding can work together to produce explainable recommendations. Future improvements include expanding the

dataset, adding inference rules for prerequisite paths, integrating GraphDB deployment screenshots, and evaluating recommendations with real student profiles.

References

Brickley, D., & Guha, R. V. (2014). RDF Schema 1.1. World Wide Web Consortium. https://www.w3.org/TR/rdf-schema/

Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., & Rudolph, S. (2012). OWL 2 Web Ontology Language primer. World Wide Web Consortium. https://www.w3.org/TR/owl2-primer/

Knublauch, H., & Kontokostas, D. (2017). Shapes Constraint Language (SHACL). World Wide Web Consortium. https://www.w3.org/TR/shacl/

Prud'hommeaux, E., & Seaborne, A. (2008). SPARQL query language for RDF. World Wide Web Consortium. https://www.w3.org/TR/rdf-sparql-query/

RDFLib Team. (2026). RDFLib documentation. https://rdflib.readthedocs.io/

W3C. (2014). RDF 1.1 Turtle: Terse RDF Triple Language. World Wide Web Consortium. https://www.w3.org/TR/turtle/

Appendix

Project files are organized in the repository as follows: ontology files are in ontology/, RDF data files are in data/sample/, SPARQL queries are in queries/, SHACL shapes are in shacl/, source code is in src/, report files are in report/, presentation files are in presentation/, and WIDOCO documentation output should be generated into docs/.